

Detección de productos: Visión Clásica vs Deep Learning

Estado del proyecto ABP — Hitos 1 a 5 completados (código). Pipeline clásico listo para entrenar.

Resumen rápido

Implementamos **dos pipelines paralelos de detección de objetos** sobre el mismo dataset de productos de supermercado:

- **Pipeline A — Visión clásica:** Selective Search + descriptores manuales (HOG, SIFT/BoVW) + SVM con kernel χ^2 .
- **Pipeline B — Deep Learning:** YOLOv8s pre-entrenado en COCO, fine-tuned sobre nuestras clases.

Ambos predicen **las mismas etiquetas** sobre **las mismas imágenes** y se evalúan con **las mismas métricas**. El entregable académico no es ganar a YOLO con el clásico — es **cuantificar la brecha y entender dónde se rompe cada paradigma**.

La regla fundamental: misma base de comparación

Para que la diferencia entre clásico y DL sea atribuible **al método** y no a otra variable, los dos pipelines comparten:

- El mismo dataset y los mismos splits train / val / test.
- Las mismas 20 etiquetas con los mismos IDs (1 a 20).
- Las mismas métricas (mAP@0.5, F1, IoU, matriz de confusión, tiempo de inferencia).
- El mismo hardware para medir tiempos (CPU local + Colab T4).

Si alguna de estas cambia, no podemos concluir que un paradigma sea mejor — podría serlo solo por haber tenido más datos o un test más fácil. Este principio guía toda la arquitectura del proyecto.

El dataset: MVTec D2S

Usamos **MVTec D2S** (Densely Segmented Supermarket): 21.000 imágenes con anotación bounding-box + máscara de 60 productos europeos. El train es "catálogo" (un objeto por imagen, fondo limpio); el val son escenas **cluttered** con varios productos juntos — el caso de uso realista.

Filtramos a un **subset de 20 clases verificadas en Mercadona España**, organizadas en grupos **visualmente confusibles**:

- 4 manzanas (Braeburn, Golden Delicious, Granny Smith, Red Boskoop) — misma forma, color sutil distinto.
- 3 pasta Reggia (elicoidali, fusilli, spaghetti) — mismo packaging azul, distinta forma a través del plástico.
- 2 Coca-Cola (clásica + light) — misma lata, color rojo vs plata.
- 2 barritas Corny (chocolate-plátano + nueces) — mismo formato, distinto sabor.
- 2 uvas (verde + morada), 2 tomates (vine + roma-vine), 2 clementinas (malla + sueltas).
- 3 singletons: zanahoria, pepino, kiwi.

Por qué grupos confusables y no clases muy distintas: distinguir manzana de coca-cola lo hace cualquier sistema. La diferencia entre clásico y DL solo se ve en distinciones finas — variedades de manzana, formas de pasta de la misma marca, sabores Corny. **Ese contraste fine-grained es donde vive el valor académico del informe.**

Qué se ha hecho (H1 a H5)

Hito	Qué	Entregable
H1	EDA + selección de las 20 clases	00_dataset_ed.ipynb, classes.yaml
H2	Splits filtrados train / val / test	data/processed/{train,val,test}.json
H3	Pipeline clásico: componentes + codebook	01_pipeline.py, descriptors/*, codebook.pkl
H4	Entrenamiento + inferencia clásica	scripts/run_classical_{train,infer}.py
H5	Hard negative mining	scripts/run_classical_hard_neg.py

Conteos resultantes: train **1650 imágenes** (2130 instancias, catálogo), val **603 imágenes** (1476 instancias, para early stopping), test **1407 imágenes** (3377 instancias, cluttered). Estratificado por clase dominante, seed fija 42 → reproducible.

Cómo funciona el pipeline clásico (alto nivel)

Flujo de un detector clásico sin redes neuronales:

```
imagen entrada
|
v
Selective Search --> ~300 cajas candidatas (sin clase aun)
|
v
Por cada caja:
  HOG --> silueta global (~1700 dims)
  SIFT + BoVW --> textura agregada (~300 dims)
  concat L2 --> vector ~2000 dims
  |
  v
20 SVMs one-vs-rest (kernel chi^2 aproximado) --> score por clase
  |
  v
Filtrado por confianza + NMS por clase
  |
  v
Hard negative mining (2 rondas):
  detecta falsos positivos -> reentrena
  |
  v
detecciones finales (bbox + clase + score)
```

En lenguaje natural:

- **Selective Search** propone dónde podría haber objetos sin saber qué son (~300 cajas por imagen).
- **HOG** describe la silueta de cada caja (perfil de bordes).
- **SIFT + BoVW** describe la textura como un histograma de "palabras visuales" aprendidas previamente.

- **SVM χ^2** decide para cada caja si es producto X, producto Y... o nada (background).
- **NMS** (Non-Maximum Suppression) elimina detecciones duplicadas que se solapan mucho.
- **Hard negative mining**: corremos el modelo entrenado sobre el train, detectamos las cajas que clasifica como producto pero no lo son, las añadimos al pool de negativos y reentrenamos. Repetimos 2-3 veces. Reduce mucho los falsos positivos.

Qué falta (H6 a H10)

Hito	Tarea	Dónde corre
H6	Pipeline B: YOLOv8s fine-tuned sobre las MISMAS 20 clases	Google Colab T4
H7	Framework de evaluación común (mAP, F1, IoU, confusión, tiempos)	CPU local
H8	Análisis de robustez (ruido, blur, iluminación, JPEG, oclusión)	CPU local
H9	Demo webcam en vivo (opcional)	CPU local
H10	Informe + presentación con tablas y figuras auto-generadas	—

Hipótesis a verificar en H7-H8: YOLOv8s va a ganar al clásico en mAP@0.5 con margen amplio, y la diferencia será **más grande en las triadas confusables** (manzanas, pasta Reggia) que en los singletons. En H8, esperamos que el clásico aguante mejor la oclusión que YOLO pero peor el ruido gaussiano y el blur.

Setup desde 0 en Windows

Pasos para llegar desde un PC vacío hasta el punto donde estamos ahora (pipeline clásico verificado, listo para entrenar).

1. Pre-requisitos

Instalar **Git for Windows** (git-scm.com) y **VSCode** (code.visualstudio.com). No hace falta instalar Python por separado — uv lo gestiona.

2. Clonar el repo

Abrir PowerShell en la carpeta donde quieras el proyecto y ejecutar:

```
git clone <URL del repo>
cd grocery-tracker
```

3. Setup automático

Instala uv si falta y sincroniza dependencias (~5-15 min, baja PyTorch ~2 GB):

```
.\scripts\setup.ps1
```

4. Descargar MVTec D2S manualmente

Ir a <https://www.mvtec.com/company/research/datasets/mvtec-d2s>, rellenar formulario y aceptar licencia. Descargar los dos archivos d2s_images_v*.tar.xz (~6 GB) y d2s_annotations_v*.tar.xz (~40 MB). Moverlos a data/raw/. Si los descomprimes manualmente, mete las carpetas images/ y annotations/ directamente en data/d2s/ y salta el paso 5.

5. Extraer el dataset

Solo si tienes los .tar.xz sin descomprimir:

```
uv run python scripts/prepare_d2s.py
```

6. Generar splits filtrados a 20 clases

Output: data/processed/{train,val,test}.json (~10s):

```
uv run python scripts/prepare_splits.py
```

7. Entrenar codebook BoVW

Vocabulario visual de K=300 palabras vía k-means sobre SIFTs de 300 imágenes train (~1 min):

```
uv run python scripts/train_codebook.py
```

8. Smoke test del pipeline clásico

Entrena con 5 imgs e infiere sobre 3 (~1-2 min). Si imprime Pipeline clasico funciona end-to-end sin crashes, todo OK:

```
uv run python scripts/test_classical_tiny.py
```

Entrenamiento e inferencia completos del pipeline clásico (~5-8 h totales, dejar de noche):

```
uv run python scripts/run_classical_train.py      # ~1-2 h
uv run python scripts/run_classical_hard_neg.py   # ~1.5-3 h
uv run python scripts/run_classical_infer.py      # ~1.5-2 h
```

Output final: reports/predictions/classical_test.json (predicciones en formato COCO listas para evaluación común en H7).

Briefing técnico completo en PROYECTO.md del repo. Detalles de cada hito en los notebooks 00_dataset_eda.ipynb, 01_class_selection.ipynb, 02_classical_dev.ipynb.